

OSSP

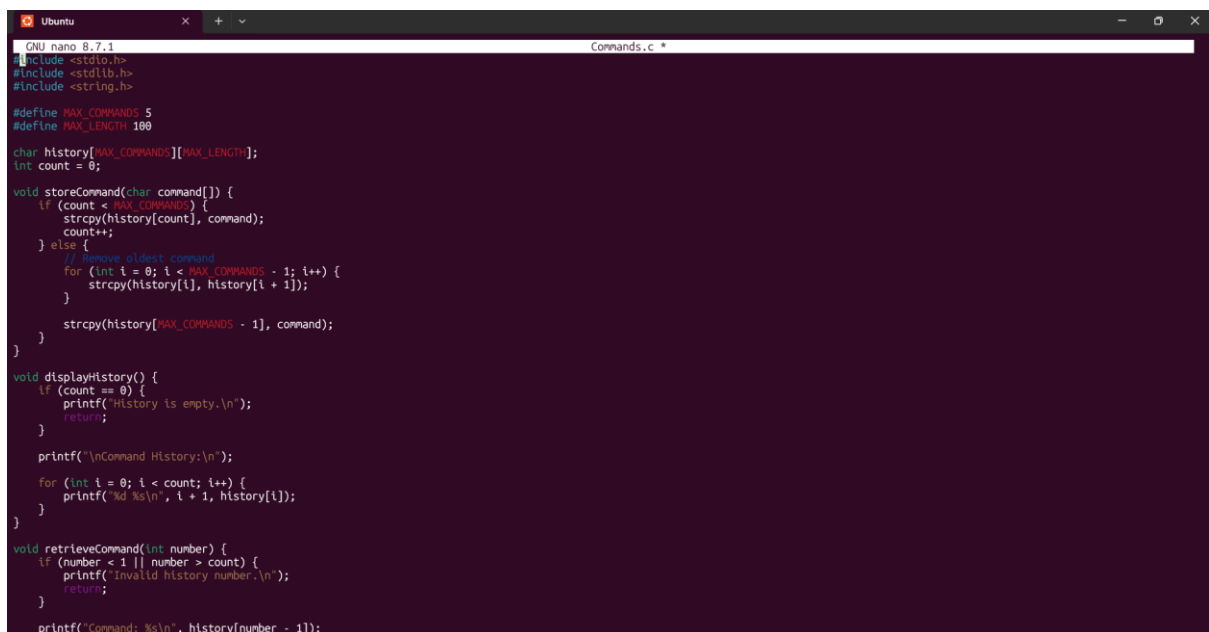
Skill Week - 6

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistenev



```
GNU nano 8.7.1 Commands.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

char history[MAX_COMMANDS][MAX_LENGTH];
int count = 0;

void storeCommand(char command[]) {
    if (count < MAX_COMMANDS) {
        strcpy(history[count], command);
        count++;
    } else {
        // Remove oldest command
        for (int i = 0; i < MAX_COMMANDS - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }
        strcpy(history[MAX_COMMANDS - 1], command);
    }
}

void displayHistory() {
    if (count == 0) {
        printf("History is empty.\n");
        return;
    }

    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }
}

void retrieveCommand(int number) {
    if (number < 1 || number > count) {
        printf("Invalid history number.\n");
        return;
    }

    printf("Command: %s\n", history[number - 1]);
}
```

GNU nano 8.7.1

```
    printf("History is empty.\n");
    return;
}

printf("\nCommand History:\n");

for (int i = 0; i < count; i++) {
    printf("%d %s\n", i + 1, history[i]);
}
}

void retrieveCommand(int number) {
    if (number < 1 || number > count) {
        printf("Invalid history number.\n");
        return;
    }

    printf("Command: %s\n", history[number - 1]);
}

int main() {
    char command[MAX_LENGTH];
    int choice, number;

    while (1) {
        printf("\n--- COMMAND HISTORY ---\n");
        printf("1. Store Command\n");
        printf("2. Display History\n");
        printf("3. Retrieve Command\n");
        printf("4. Exit\n");
        printf("Enter choice: ");

        scanf("%d", &choice);
        getchar();

        switch (choice) {

            case 1:
                printf("Enter command: ");

                fgets(command, MAX_LENGTH, stdin);

                command[strcspn(command, "\n")] = '\0';
```

```
GNU nano 8.7.1 Commands.c *
printf("2. Display History\n");
printf("3. Retrieve Command\n");
printf("4. Exit\n");
printf("Enter choice: ");

scanf("%d", &choice);
getchar();

switch (choice) {

    case 1:
        printf("Enter command: ");

        fgets(command, MAX_LENGTH, stdin);

        command[strcspn(command, "\n")] = '\0';

        storeCommand(command);

        printf("Command stored.\n");
        break;

    case 2:
        displayHistory();
        break;

    case 3:
        printf("Enter history number: ");
        scanf("%d", &number);

        retrieveCommand(number);
        break;

    case 4:
        exit(0);

    default:
        printf("Invalid choice.\n");

}

return 0;
}
```

Output:-

```
nocturnal@nocturnal:~/producer-consumer$ ./Commands

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 1
Enter command: lws
Command stored.

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 2

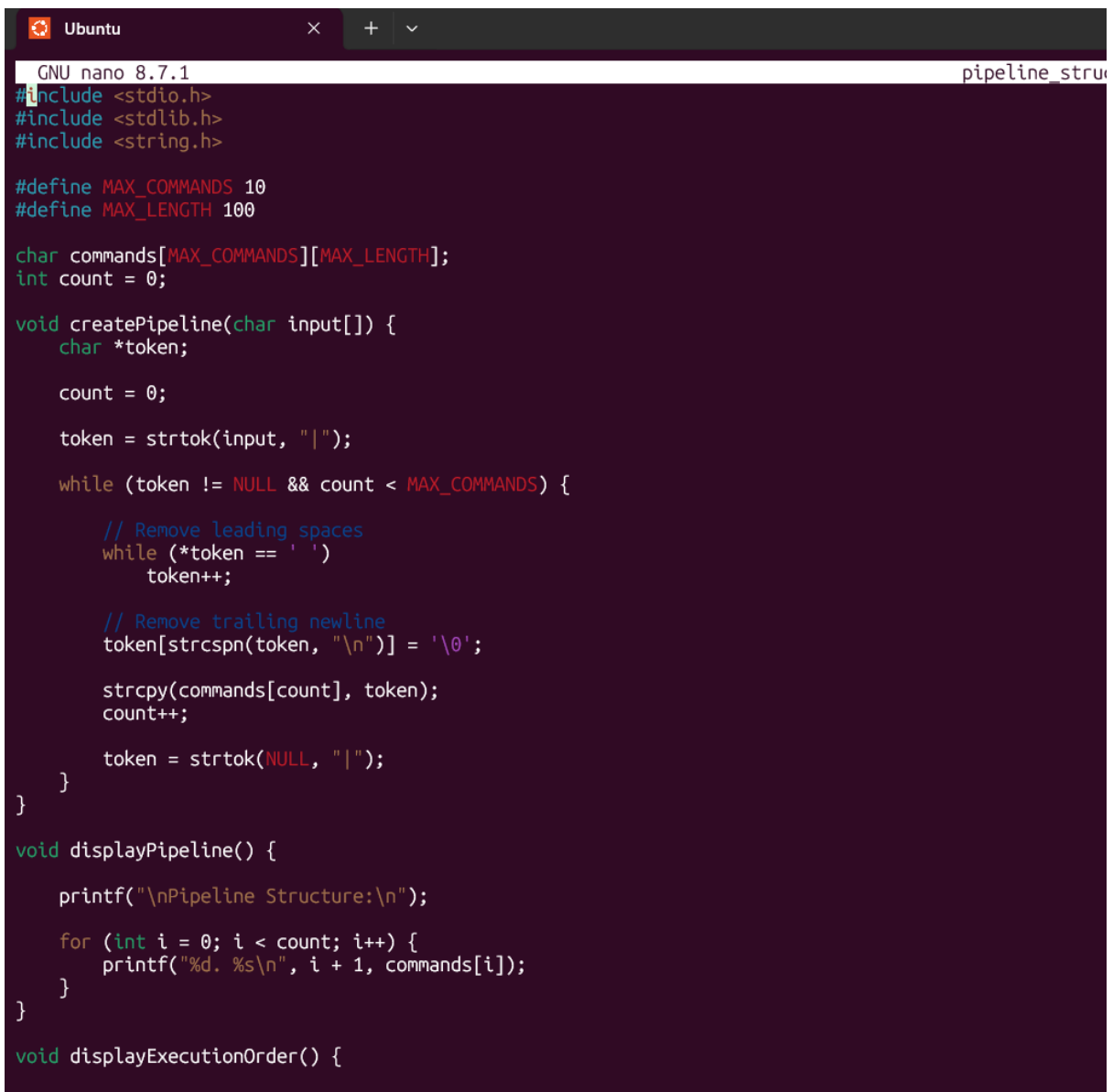
Command History:
1 lws

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 3
Enter history number: 1
Command: lws

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 4
nocturnal@nocturnal:~/producer-consumer$
```

Task-2:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations



```
GNU nano 8.7.1 pipeline_stru
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 100

char commands[MAX_COMMANDS][MAX_LENGTH];
int count = 0;

void createPipeline(char input[]) {
    char *token;

    count = 0;

    token = strtok(input, "|");

    while (token != NULL && count < MAX_COMMANDS) {

        // Remove leading spaces
        while (*token == ' ')
            token++;

        // Remove trailing newline
        token[strcspn(token, "\n")] = '\0';

        strcpy(commands[count], token);
        count++;

        token = strtok(NULL, "|");
    }
}

void displayPipeline() {

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < count; i++) {
        printf("%d. %s\n", i + 1, commands[i]);
    }
}

void displayExecutionOrder() {
```

```
Ubuntu × + ▾
GNU nano 8.7.1 pipeli
    }
    token = strtok(NULL, "|");
}

void displayPipeline() {
    printf("\nPipeline Structure:\n");

    for (int i = 0; i < count; i++) {
        printf("%d. %s\n", i + 1, commands[i]);
    }
}

void displayExecutionOrder() {
    printf("\nExecution Order:\n");

    for (int i = 0; i < count; i++) {
        printf("%d -> %s\n", i + 1, commands[i]);
    }
}

int main() {
    char input[MAX_LENGTH * MAX_COMMANDS];

    printf("Enter pipeline command separated by |:\n");
    printf("Example: ls -l | grep txt | wc -l\n");

    printf("Pipeline: ");

    fgets(input, sizeof(input), stdin);

    createPipeline(input);

    if (count == 0) {
        printf("Invalid pipeline.\n");
        return 1;
    }

    displayPipeline();

    displayExecutionOrder();

    printf("\nPipeline execution completed successfully.\n");

    return 0;
}
```

Output –

```
nocturnal@nocturnal:~/producer-consumer$ nano pipeline_structure.c
nocturnal@nocturnal:~/producer-consumer$ gcc pipeline_structure.c -o pipeline_structure
nocturnal@nocturnal:~/producer-consumer$ ./pipeline_structure
Enter pipeline command separated by |:
Example: ls -l | grep txt | wc -l
Pipeline: ls -l | wc -l

Pipeline Structure:
1. ls -l
2. wc -l

Execution Order:
1 -> ls -l
2 -> wc -l

Pipeline execution completed successfully.
nocturnal@nocturnal:~/producer-consumer$
```

Task 3 - :

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
Ubuntu × + ▾
GNU nano 8.7.1 pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    printf("pipe created successfully.\n");

    p1 = fork();

    if (p1 < 0) {
        perror("fork");
        return 1;
    }

    if (p1 == 0) {

        // First command: ls
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    p2 = fork();

    if (p2 < 0) {
        perror("fork");
        return 1;
    }

    if (p2 == 0) {

        // Second command: wc -l
```

```
Ubuntu
GNU nano 8.7.1
if (p1 == 0) {

    // First command: ls
    close(fd[0]);

    dup2(fd[1], STDOUT_FILENO);

    close(fd[1]);

    execlp("ls", "ls", NULL);

    perror("execlp");
    exit(1);
}

p2 = fork();

if (p2 < 0) {
    perror("fork");
    return 1;
}

if (p2 == 0) {

    // Second command: wc -l
    close(fd[1]);

    dup2(fd[0], STDIN_FILENO);

    close(fd[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("execlp");
    exit(1);
}

// Parent process
close(fd[0]);
close(fd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("data flow completed successfully.\n");

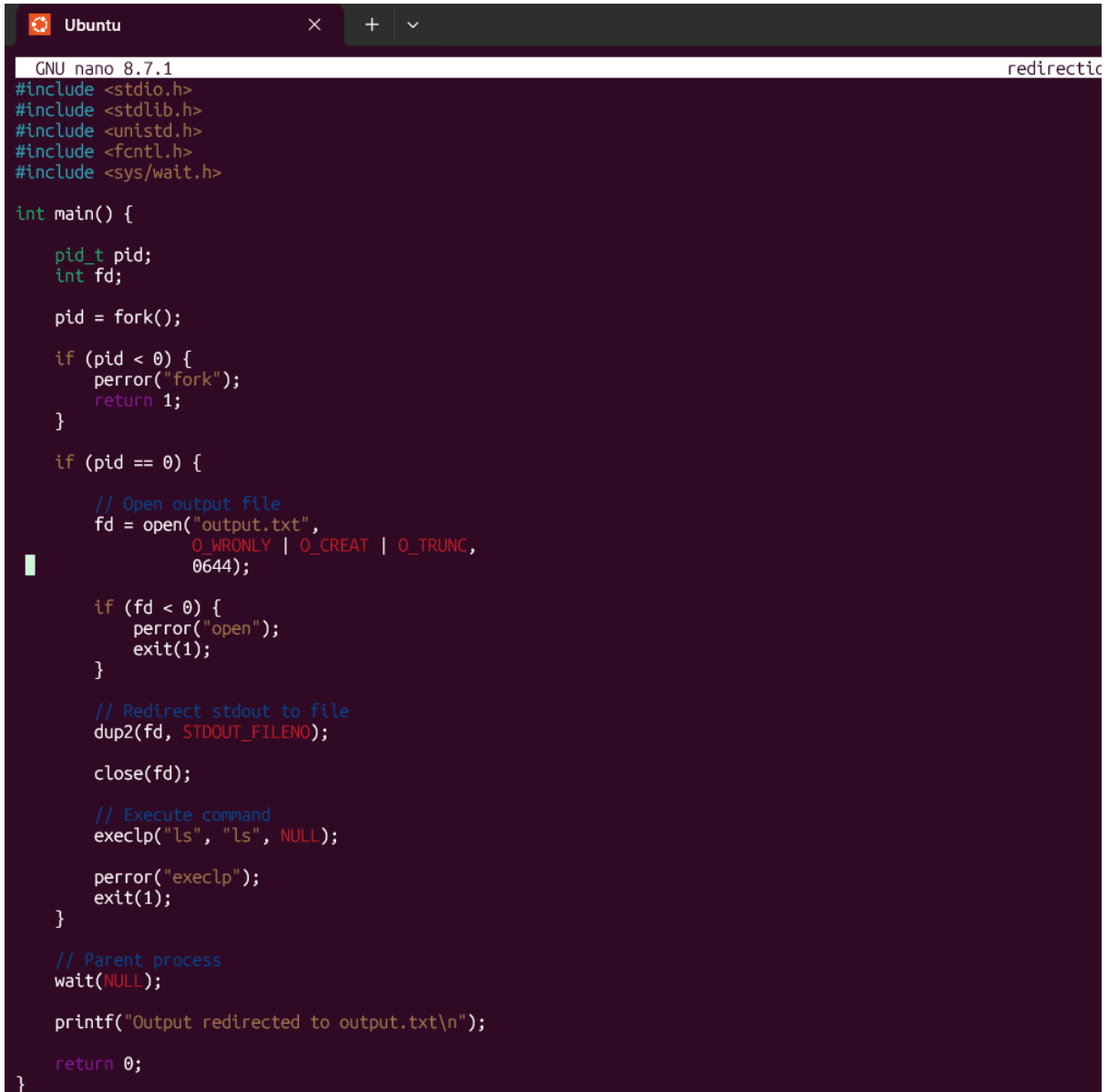
return 0;
}
```

Output - :

```
Ubuntu
nocturnal@nocturnal:~/producer-consumer$ nano pipeline.c
nocturnal@nocturnal:~/producer-consumer$ gcc pipeline.c -o pipeline
nocturnal@nocturnal:~/producer-consumer$ gcc pipeline.c -o pipeline
nocturnal@nocturnal:~/producer-consumer$ ./pipeline
pipe created successfully.
16
data flow completed successfully.
nocturnal@nocturnal:~/producer-consumer$
```

Task 4 -:

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines



```
GNU nano 8.7.1                                redirectio
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int fd;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // Open output file
        fd = open("output.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

        if (fd < 0) {
            perror("open");
            exit(1);
        }

        // Redirect stdout to file
        dup2(fd, STDOUT_FILENO);

        close(fd);

        // Execute command
        execlp("ls", "ls", NULL);

        perror("execlp");
        exit(1);
    }

    // Parent process
    wait(NULL);

    printf("Output redirected to output.txt\n");

    return 0;
}
```

Output-:

Ubuntu

×

+

▼

```
nocturnal@nocturnal:~/producer-consumer$ nano redirection.c
nocturnal@nocturnal:~/producer-consumer$ gcc redirection.c -o redirection
nocturnal@nocturnal:~/producer-consumer$ gcc redirection.c -o redirection
nocturnal@nocturnal:~/producer-consumer$ ./redirection
Output redirected to output.txt
nocturnal@nocturnal:~/producer-consumer$
```